Sign In
Get started

Published in Towards Data Science

You have **2** free member-only stories left this month.
Sign up for Medium and get an extra one

Bharath K Follow

Dec 22, 2020 · 14 min read · Member-only · Listen

GETTING STARTED

Everything You Need To Know About Jupyter Notebooks

Breaking down everything about Jupyter Notebooks ranging from the menus, to running the cells, and the various shortcuts options available

Photo by Christopher Gower on Unsplash

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. Its uses include data cleaning and transformation, numerical simulation, statistical modeling, data visualization, machine learning, and much more.

"notebook" term can colloquially make reference to many different entities, mainly the Jupyter web application, Jupyter Python web server, or Jupyter document format depending on context.

According to the official website of Jupyter, Project Jupyter exists to develop open-source software, open-standards, and services for interactive computing across dozens of programming languages.

Jupyter Book is an open-source project for building books and documents from computational material. It allows the user to construct the content in a mixture of Markdown, an extended version of Markdown called MyST, Maths & Equations using MathJax, Jupyter Notebooks, reStructuredText, the output of running Jupyter Notebooks a 306 1 ltiple output formats can be produced (currently single files, multipage HTML web pages and PDF files).

Now that we have a brief understanding of the concept of Project Jupyter and the Jupyter Notebooks, and we have established that these Notebooks are quite revolutionary in the modern ages, let us proceed to understand the more in-depth details of this amazing interactive environment in the next sections.

installed in the Anaconda package, and you don't have to install this on your own.

One of the requirements here is Python, either Python 3.3 or greater or Python 2.7. The general recommendation is that you use the Anaconda distribution to install both Python and the notebook application. The advantage of Anaconda is that you have access to over 720 packages that can easily be installed with Anaconda's conda, a package, dependency, and environment manager.

I would highly recommend checking out the following two links for the installation of Anaconda on your system. You can access them from here and here.

However, if you are not interested in installing the Anaconda package manager, and you want to simply install the Jupyter Notebook the Pythonic way, make sure you have downloaded the latest Python Version (which is 3.9 at the time of writing of this article), and then proceed to issue the following command in the command prompt or PowerShell.

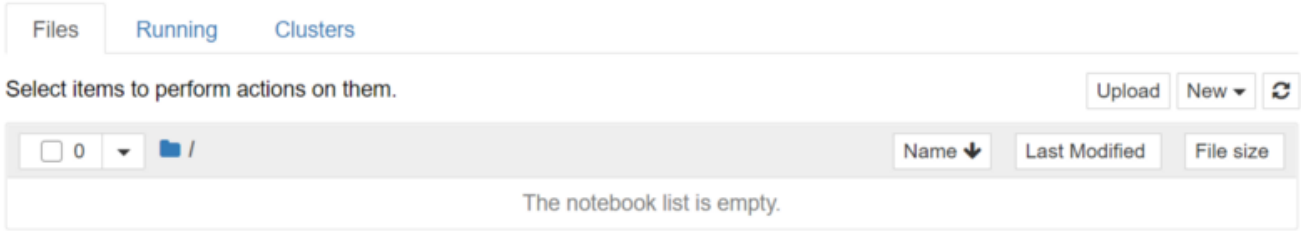
```
pip install jupyter
```

in your selected web browser with a local hosting URL, which is labeled as http://localhost:8888.

Details Of The Jupyter Notebook Environment

To follow with this section of the tutorial, I would highly recommend all of you have your Jupyter Notebooks open side by side. This will help you to follow along, analyze, and grasp most of the essentials of the Jupyter Notebook more easily.

After launching the Jupyter Notebook on your local host web browser, you should be directed to a page that looks as follows —

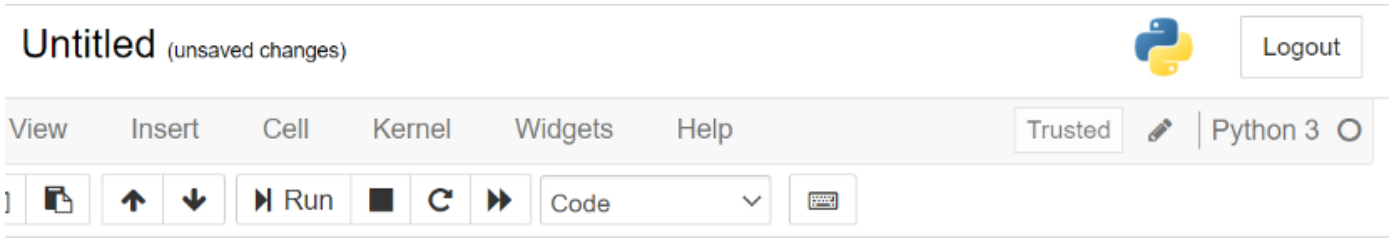


Screenshot By Author

> **Note:** All images are zoomed in to 175% for better clarity.

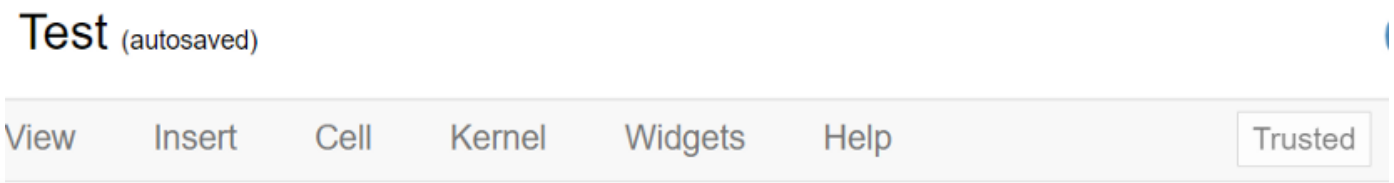
change this by simply clicking on the New button and selecting the Python 3 option from the menu that is provided upon your click.

After clicking on the Python 3 option from the New dropdown menu, you will be directed to a new Notebook that is created. It will look similar to the screenshot below.



Screenshot By Author

The first thing we will do before looking into the various menu options available in the Jupyter Notebook is changing our title to the desired name of our choice. I will be labeling it as **Test**, and it should look something similar to the image shown below.

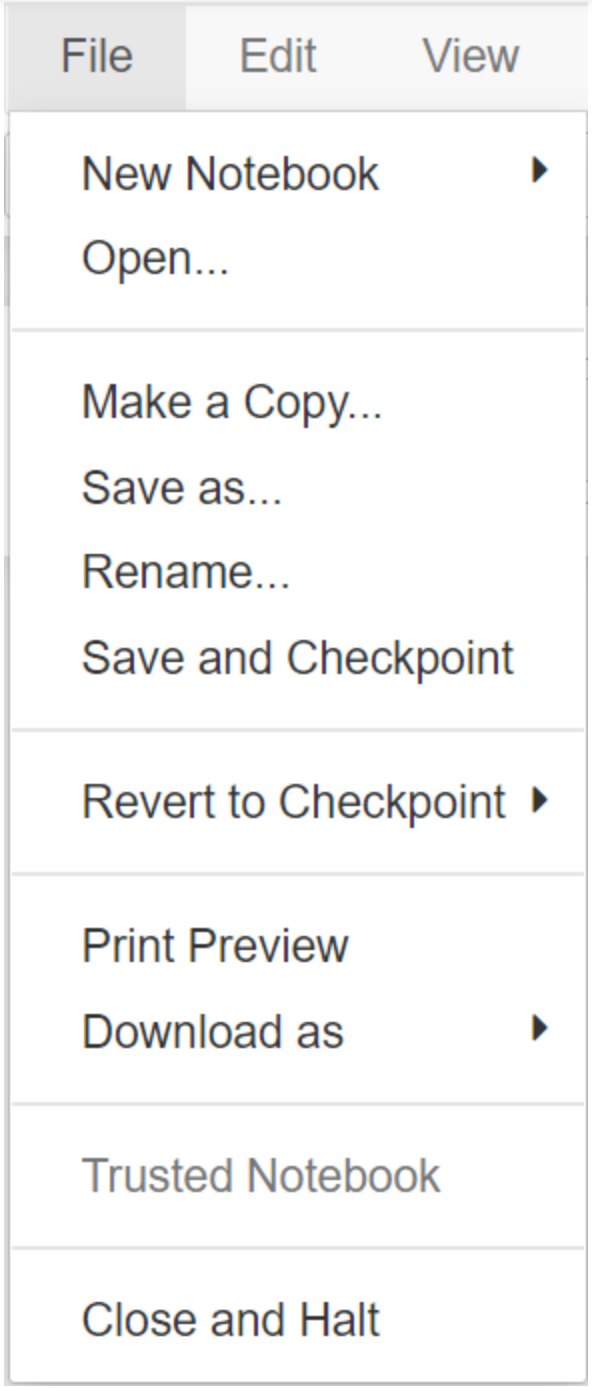


Upon cross checking the local host URL that was launched, you will now notice that there is a new Jupyter Notebook file that is created in the specific directory.

Screenshot By Author

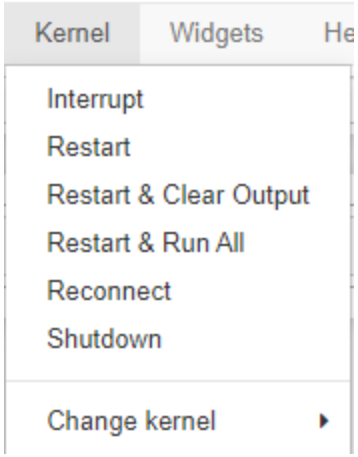
You can add any number of Python or other files in the directory as you desire, and it will be constantly updated. With this basic introduction to the Jupyter Notebook platform, let us proceed to understand most of the significant menu options available.

> **Note:** In this section, I will be focusing mostly on only the more important menu options that are usually used on a consistent basis. I will skip other additional commands that are not frequently used.


File
Edit
View
New Notebook
Open...
Make a Copy...
Save as...
Rename...
Save and Checkpoint
Revert to Checkpoint
Print Preview
Download as
Trusted Notebook
Close and Halt

1. **New Notebook:** The name is self-explanatory as it allows you to create multiple new Notebooks for your Python or Data Science projects.
2. **Open:** This command will redirect you to your local host and you can open other files from there.
3. **Make a copy:** You can create duplicate copies of your Jupyter Notebook that will be useful for trying various experiments with your Data Science Projects.
4. **Save as:** Used to save your Notebook with a specific path.
5. **Rename:** You can use this command to Rename your Notebook or simply double click on the title header instead for performing the same action.
6. **Save and Checkpoint:** As the name suggests it saves your progress and checkpoints the file accordingly. The shortcut key **ctrl + s** will perform an equivalent action.
7. **Revert to Checkpoint:** If you ever mess up with while experimenting with the Jupyter Notebook, this is an useful command to revert back to your previous saved state and nullify your mistakes.
8. **Print Preview:** Used to get only the code part of the Jupyter Notebook for a clean print.
9. **Download as:** Allows you to download your Jupyter Notebook in various formats including HTML, IPython, .py format, etc., among many other options.

Kernel:



Screenshot By Author

The kernel is a computer program at the core of a computer's operating system that has complete control over everything in the system. It is the "portion of the operating system code that is always resident in memory", and facilitates interactions between hardware and software components.

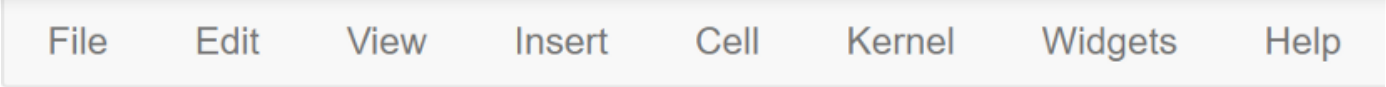
A notebook kernel is a "computational engine" that executes the code contained in a Notebook document. The ipython kernel, referenced in this guide, executes python code. Kernels for many other languages exist (official kernels).

When you open a Notebook document, the associated kernel is automatically

results. Depending on the type of computations, the kernel may consume significant CPU and RAM. Note that the RAM is not released until the kernel is shut-down.

Let us proceed to analyze the various functions in the Kernel menu icon that will be useful for Data Science projects.

1. **Interrupt:** Used to stop the execution or running of a particular cell. This command is useful when you have reached a desired result within a specific number of epochs or in case you made an error, and you realize this while running the code.
2. **Restart:** Useful to restart the Kernel of the Notebook.
3. **Restart and Clear Output:** Used to restart the Kernel of the Notebook and reset all the cells that were run previously.
4. **Restart and Run all:** Used to restart the Kernel of the Notebook, and reset all the cells that were run previously, and finally re-run through all the cells of the Notebook.
5. **Reconnect:** Used to reconnect to a dead kernel which might occur at times due to a lack of memory.
6. **Shutdown:** Used to shutdown the current working Kernel of the Notebook.
7. **Change kernel:** Allows you to switch Kernels.

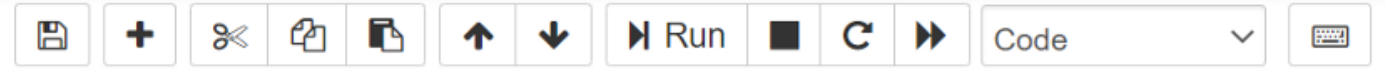


Screenshot By Author

Since we have already discussed the “File” and “Kernel options in the main menu bar in detail, let us briefly look at the other options and the functionalities they offer us.

- **Edit:** Used to edit the cells and perform functionalities like adding, cutting, moving, or deletion of cell blocks.
- **View:** Used to change the appearance of the Notebook as per the users desire.
- **Insert:** Used to insert a cell block above or below a specific selected cell.
- **Cell:** Used to run the various cells or select a specific cell type.
- **Widgets:** You can use widgets to build interactive GUIs for your notebooks. Widgets are eventful python objects that have a representation in the browser, often as a control like a slider, textbox, etc.
- **Help:** Provides help and also allows you to view the shortcut options available in the Jupyter Notebook.

Command Icon Palette:



Screenshot By Author

1. **Save and Checkpoint:** As the name suggests it saves your progress and checkpoints the file accordingly. The shortcut key **ctrl + s** will perform an equivalent action.
2. **Insert Cell Below:** Adds a new cell below.
3. **Cut Selected Cells:** Cuts the selected cells.
4. **Copy Selected Cells:** Copies the selected cells.
5. **Paste Selected Cells:** Pastes the selected cells.
6. Move the selected cells upward.
7. Move the selected cells downward.
8. **Run:** Runs the particular selected cell.
9. **Interrupt:** Used to stop the execution or running of a particular cell.
10. **Restart:** Useful to restart the Kernel of the Notebook.
11. **Restart and Run all:** Used to restart the Kernel of the Notebook, and reset all the cells that were run previously, and finally re-run through all the cells of the Notebook.
12. **Change Cell Type:** Allows you the option to convert the type of cell into

Running The Code Cells:

```
In [1]: print("Hello World!")
        Hello World!

In [2]: a = "Hi"

In [3]: a
Out[3]: 'Hi'

In [4]: 5+6
Out[4]: 11

In [ ]:
```

Screenshot By Author

You can run the notebook document step-by-step (one cell a time) by pressing *crtl* + *enter* for running the particular cell or *shift* + *enter* to run the current cell as well create a new cell below it. You can run the whole notebook in a single step by clicking on the menu *Cell -> Run All*.

The above image is a representation of how each of these cells can run as separate code blocks. In the first code cell you can notice that we have used the print statement for printing a basic “Hello World!” program.

new code cell to receive the held value. The code cells [2] and [3] are a representation of the same.

Finally, the code cells can also be used as a calculator to perform a variety of tasks ranging from addition, subtraction, multiplication, division, power (**), and many more functions. You can also utilize these code blocks for programming your python or Data Science projects.

Markdowns:

Text can be added to Jupyter Notebooks using Markdown cells. You can change the cell type to Markdown by using the `Cell` menu, the toolbar, or the key shortcut `m`. Markdown is a popular markup language that is a superset of HTML.

Markdowns are a way to make your Jupyter Notebooks more presentable and help to make the viewers or readers a more concise understanding of your code. You can add headers, images, widgets, bullet points, ordered lists, hyperlinks, and so much more.

More details about its specifications can be found here. To learn more about

Sign In
Get started

Quick Guide To Shortcuts In Jupyter Notebook

Below is a image representation of most of the essential shortcut commands of the Jupyter Notebook.

Sign In Get started

Keyboard shortcuts

Command Mode (press Esc to enable)

Edit Shortcuts

- F : find and replace
- Ctrl-Shift-F : open the command palette
- Ctrl-Shift-P : open the command palette
- Enter : enter edit mode
- P : open the command palette
- Shift-Enter : run cell, select below
- Ctrl-Enter : run selected cells
- Alt-Enter : run cell and insert below
- Y : change cell to code
- M : change cell to markdown
- R : change cell to raw
- 1 : change cell to heading 1
- 2 : change cell to heading 2
- 3 : change cell to heading 3
- 4 : change cell to heading 4
- 5 : change cell to heading 5
- 6 : change cell to heading 6
- K : select cell above
- Up : select cell above
- Down : select cell below
- J : select cell below
- Shift-K : extend selected cells above
- Shift-Up : extend selected cells above
- Shift-Down : extend selected cells below
- Shift-J : extend selected cells below
- A : insert cell above
- B : insert cell below
- X : cut selected cells
- C : copy selected cells
- Shift-V : paste cells above
- V : paste cells below
- Z : undo cell deletion
- D , D : delete selected cells
- Shift-M : merge selected cells, or current cell with cell below if only one cell is selected
- Ctrl-S : Save and Checkpoint
- S : Save and Checkpoint
- L : toggle line numbers
- O : toggle output of selected cells
- Shift-O : toggle output scrolling of selected cells
- H : show keyboard shortcuts
- I , I : interrupt the kernel
- 0 , 0 : restart the kernel (with dialog)
- Esc : close the pager

Close

I have discussed most of the necessary shortcuts throughout the course of this article. However, if you want to learn and explore more about the various shortcut options available, then click on the help and open shortcut command options for a more detailed guide to these shortcut choices in the Jupyter Notebook.

Summarizing The Pros and Cons:

We have understood some of the details and in-depth mechanisms of Jupyter Notebooks. We can conclude that these Notebooks are absolutely amazing in performing tasks related to visualizations, cleansing of data, and any projects related to data science or Python in general.

To quickly brief up some of the pros and cons of Jupyter Notebooks, they can be considered as provided below.

Pros:

- Best Platform for getting started with data science.
- Easy to share notebooks and visualizations.
- Availability of markdowns and other additional functionalities.

If you want a concise guide of more than ten awesome Python editors, then I would highly recommend all of you to check out the following article, whose link is provided below. It has a brief understanding of some awesome editor choices and alternative options that you can use as your IDE.

A Concise Guide of 10+ Awesome Python Editors and How To Choose Which Editor Suits You The Best...

A concise guide to help you choose between the various development environments for python

towardsdatascience.com

Brief Understanding Of The Google Colab

Jupyter Notebooks are more popular than ever right now and is used by almost every major company for its interactive environment and the fact that sharing these notebooks is very easy. An example of this is the project developed by Google known as the Google Colaboratory.

Colaboratory (also known as Colab) is a free Jupyter notebook environment that runs in the cloud and stores its notebooks on Google Drive. Colab was originally an internal Google project; an attempt was made to open source all

"Open in Colab" Google Chrome extension, but this eventually ended, and Colab development continued internally.

As of October 2019, the Colaboratory UI only allows you to create notebooks with Python 2 and Python 3 kernels; however, if you have an existing notebook whose kernelspec is IR or Swift, that will work, since both R and Swift are installed in the container. Julia language can also work on Colab (with e.g. Python and GPUs; Google's tensor processing units also work with Julia on Colab.

Overall, Jupyter Notebooks are a fantastic piece of work for every Data Science enthusiast and something every lover of this field must explore. Whether you love it or hate it, it is undeniable that this workspace is an innovation for the upcoming years due to its high interactivity, compactness, and easily shareable capabilities.

Additional Information:

Project Jupyter is a nonprofit organization created to "develop open-source software, open-standards, and services for interactive computing across dozens of programming languages". Spun off from IPython in 2014 by

Project Jupyter's name is a reference to the three core programming languages supported by Jupyter, which are Julia, Python and R, and also a homage to Galileo's notebooks recording the discovery of the moons of Jupiter. Project Jupyter has developed and supported the interactive computing products Jupyter Notebook, JupyterHub, and JupyterLab.

A Jupyter Notebook document is a JSON document, following a versioned schema, containing an ordered list of input/output cells which can contain code, text (using Markdown), mathematics, plots and rich media, usually ending with the ".ipynb" extension.

A Jupyter Notebook can be converted to a number of open standard output formats (HTML, presentation slides, LaTeX, PDF, ReStructuredText, Markdown, Python) through "Download As" in the web interface, via the nbconvert library or "jupyter nbconvert" command line interface in a shell.

To simplify visualisation of Jupyter notebook documents on the web, the nbconvert library is provided as a service through NbViewer which can take a URL to any publicly available notebook document, convert it to HTML on the fly and display it to the user.

Photo by Jasper Boer on Unsplash

Conclusion:

In this article, we understand almost all of the details pertaining to Jupyter Notebooks. We learned about the “Project Jupyter” and how the Jupyter Notebook, with its high-level interactivity and easily shareable features, is making an evolutionary change for data science enthusiasts. Despite having a

The Jupyter Notebook is an absolutely fantastic option to get started with data science and machine learning. These Notebooks can be shared with anyone and helps to collaborate code more efficiently and effectively. I would highly recommend using the Jupyter Notebook as well because you can use each code block separately and you also have the option to use markdowns. It is widely used in lots of profitable companies.

I hope this concise guide was able to help out the viewers to understand the concept of Jupyter Notebooks and how to make better use of this IDE. Feel free to let me know if I missed out on something that you think I should have covered in this article.

There are so many other IDE options to choose from, lots of them, to be honest! But don't be overwhelmed. I would like to reiterate that there is no right or wrong choice of the IDE. In the end, choose the editor that you feel you are the most comfortable with and enjoy using the most.

Check out some of my other articles that you might enjoy reading!

Mastering Python Lists For Programming!

Detailed understanding of all the concepts of lists Required for Programming. Why use lists for Data Science and...

10 Wrong Reasons To Become A Data Scientist

Analyzing The 10 Wrong Reasons For Pursuing Data Science To Become A Data Scientist

towardsdatascience.com

Understanding The Binary Search Algorithm In Python

Detailed understanding of the working of the binary search algorithm and its implementation in python.

towardsdatascience.com

Mastering Python Lists For Programming!

Detailed understanding of all the concepts of lists Required for Programming. Why use lists for Data Science and...

towardsdatascience.com

10 Steps To Master Python For Data Science

The 10 step procedure to become a master in Python for Data Science and Machine Learning

towardsdatascience.com

Thank you all for sticking on till the end. I hope all of you enjoyed reading the article. Wish you all a wonderful day!

References:

1. Wikipedia
2. Data Camp
3. Jupyter / Ipython Documentation
4. Official Jupyter Website

Enjoy the read? Reward the writer.[Beta]

Your tip will go to Bharath K through a third-party platform of their choice, letting them know you appreciate their story.

Give a tip

Sign up for The Variable

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter